

Walls and Gates Notes

Authored by: Arpit Raj , LNMIIT Jaipur , July 22, 2026

Grid Setup & Rules

Each cell can have one of three values:

-1

→ Wall

0

→ Gate

INF

→ Empty Room

Here,

INF = 2147483647

which is INT_MAX. For every empty room, fill it with the distance to its nearest gate.

If a room cannot reach any gate, leave it as INF.

Imagine Aadya opens every gate simultaneously.

Every second she walks one step.

Walls stop her.

When Aadya first reaches a room, that room has found its nearest gate.

Suppose two Aadyas (i.e. that Aadya is so special that there can't be a second one, but let's just assume) start from two different gates.

The room will always be reached first by the nearest Aadya.

Wrong Approach (Brute Force)

For every room

Run BFS



Find nearest gate

If there are $n*m$ rooms,

Time $O((n*m)^2)$

Too slow.

```
class Solution {
public:
    void wallsAndGates(vector<vector<int>>& rooms) {

        int n=rooms.size();
        int m=rooms[0].size();

        vector<pair<int,int>> dir={{1,0},{-1,0},{0,1},{0,-1}};

        queue<pair<int,int>> q;

        //Push every gate
        for(int i=0;i<n;i++){

            for(int j=0;j<m;j++){

                if(rooms[i][j]==0){

                    q.push({i,j});

                }

            }

        }

        while(!q.empty()){
```

```

        int r=q.front().first;
        int c=q.front().second;
        q.pop();

        for(auto it:dir){

            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nr<n && nc>=0 && nc<m){

                //Only visit empty rooms
                if(rooms[nr][nc]==INT_MAX){

                    rooms[nr][nc]=rooms[r][c]+1;

                    q.push({nr,nc});

                }

            }

        }

    }

};

```

Why don't we need a visited array?

Initially

INT_MAX

means

Not visited.

Once Aadya reaches a room,

`rooms[nr][nc]=distance;`

Now it's no longer INT_MAX.

So it automatically becomes visited.
That's exactly why Multi-Source BFS works.